

CS2601 Linear and Convex Optimization

Homework 11 Solution

Due: 2021.12.16

For this assignment, you should submit a **single** pdf file as well as your source code (.py or .ipynb files). The pdf file should include all necessary figures, the outputs of your Python code, and your answers to the questions. Do NOT submit your figures in separate files. Your answers in any of the .py or .ipynb files will NOT be graded.

Complete the implementation of projected gradient descent method in `proj_gd.py`.

1. **Lasso.** Implement the projection onto ℓ_1 ball. Use projected gradient descent to solve the Lasso problem on slide 12 of §11,

$$\begin{aligned} \min_{\mathbf{w}} \quad & \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 \\ \text{s.t.} \quad & \|\mathbf{w}\|_1 \leq t \end{aligned} \quad \mathbf{X} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 3 \\ 2 \\ 2 \end{bmatrix}, \quad t = 1$$

Use the initial point $\mathbf{w}_0 = (-1, 0.5)^T$ and a step size of your choice. Report the solution and the number of iterations. Plot the trajectory of $\mathbf{w}_k$ and the gap $f(\mathbf{w}_k) - f(\mathbf{w}^*)$.

Proof. For the plots, see Figure 1.

number of iterations: 69

solution: [9.99999997e-01 2.52227830e-09]

value: 4.5

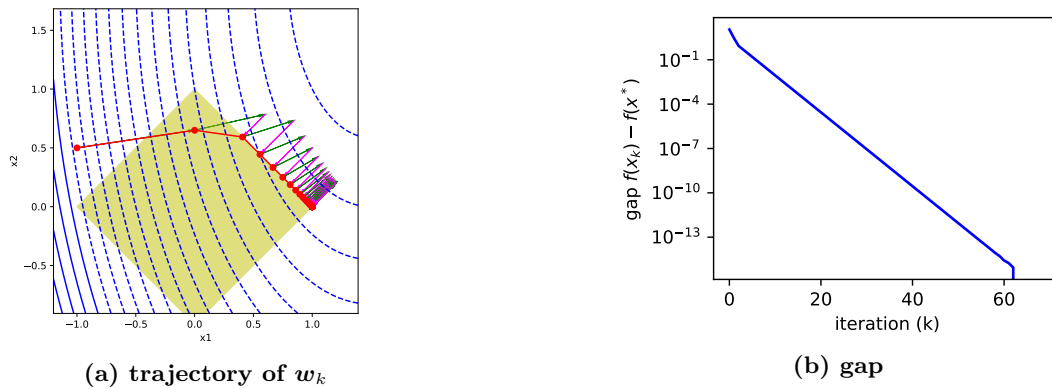


Figure 1: $\mathbf{w}_0 = (-1, 0.5)^T$

□

A possible implementation.

```
def proj_gd(fp, proj, x0, stepsize, tol=1e-5, maxiter=100000):
    x_traces = [np.array(x0)]
    y_traces = []
    x = np.array(x0)

    for k in range(maxiter):
        y = x - stepsize * fp(x)
        x_next = proj(y)

        if np.linalg.norm(x_next - x) < stepsize * tol:
            break

        x = x_next

        y_traces.append(y)
        x_traces.append(np.array(x))

    return x_traces, y_traces

def proj_norm1(x, radius=1):
    x_norm1 = np.linalg.norm(x, ord=1)

    if x_norm1 <= radius:
        return x

    y = np.flip(np.sort(np.abs(x)))
    mu = (np.cumsum(y) - radius)/np.arange(1, len(y)+1)
    k = np.where(mu <= y)[0][-1]
    return np.sign(x) * np.maximum(np.abs(x) - mu[k], 0)
```

2. Let $\mathbf{x} \in \mathbb{R}^3$. Consider

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) = e^{x_1} + e^{2x_2} + e^{2x_3} \\ \text{s.t.} \quad & x_1 + x_2 + x_3 = 1 \end{aligned} \tag{1}$$

- (a). Solve problem (1) by the Lagrange multiplier method. Show the optimal solution $\mathbf{x}^*$, the Lagrange multiplier λ^* and the optimal value f^* .
- (b). Implement the projection onto an affine space. Note some matrix/vector multiplications can be pre-computed and reused in each iteration. Use projected gradient descent to solve (1) with initial points $\mathbf{x}_0 = (0, 0, 0)^T$. Show the output.

Solution. (a). The Lagrangian is

$$\mathcal{L}(\mathbf{x}, \lambda) = e^{x_1} + e^{2x_2} + e^{2x_3} + \lambda(x_1 + x_2 + x_3 - 1)$$

The optimality condition is

$$\begin{cases} e^{x_1} + \lambda = 0 \\ 2e^{2x_2} + \lambda = 0 \\ 2e^{3x_3} + \lambda = 0 \\ x_1 + x_2 + x_3 - 1 = 0 \end{cases}$$

Thus

$$\begin{cases} x_1 = \log(-\lambda) \\ x_2 = \frac{1}{2} \log(-\lambda) - \frac{1}{2} \log 2 \\ x_3 = \frac{1}{2} \log(-\lambda) - \frac{1}{2} \log 2 \end{cases}$$

Substitution into the constraint yields,

$$2 \log(-\lambda) = 1 + \log 2 \implies \lambda^* = -\sqrt{2e}$$

$$\begin{cases} x_1^* = \frac{1+\log 2}{2} \\ x_2^* = \frac{1-\log 2}{4} \\ x_3^* = \frac{1-\log 2}{4} \end{cases}$$

and

$$f^* = -2\lambda^* = 2\sqrt{2e}$$

(b). The output is shown below.

```
number of iterations: 52
solution: [0.84657359 0.07671321 0.07671321]
value: 4.663287963194248
```

A possible implementation is given below.

```
B = A.T@np.linalg.inv(A@A.T)
P = np.eye(A.shape[1]) - B@A
c = B@b
```

```
def proj(x):
    return P@x + c
```

□